<김기환 / >

Summary

프론트엔드 엔지니어 김기환입니다. 리디(RIDI)에서 대규모 콘텐츠 플랫폼의 성능 최적화, Cross-device 안정성 개선, Server-Driven UI 전환을 담당했습니다. 현재는 두나무 자회사 Levvels에서 AI 기반 브라우저 에이전트, 생성형 UI, 커머스 프론트엔드 경험을 만들고 있습니다. 특히 내부 운영 도구에서는 생성형 UI를 단순한 화면 생성이 아니라 여러 시스템을 연결하고 조작하는 Agent 실행 레이어로 확장하는 작업을 진행했습니다. 티맥스엔터프라이즈에서는 AutoML 플랫폼과 AI 기능을 서비스 UI로 전달하는 인터페이스를 다뤘고, 석사 과정에서는 Visual Analytics와 추천 시스템의 AI 기반 UI를 연구했습니다. 이러한 연구적 사고를 실무 개발에 접목해 문제 정의, 인터랙션 설계, 프론트엔드 구조 개선을 함께 다룹니다.

juljin1875@gmail.com LinkedIn GitHub

Experience

Levvels

프론트엔드 엔지니어
2025.07–현재

내부 운영 자동화와 크리에이터 커머스 서비스의 프론트엔드를 다룹니다. 운영 도구에서는 자연어 요청을 여러 시스템 조작 흐름으로 변환하는 Agent 기반 인터페이스를 검토하고 구현했습니다.

React · Next.js · TypeScript · Browser Agent · Generative UI · LLM Prompt Engineering · Amplitude · ChannelTalk

PageAgent 기반 내부 운영 자동화

2026.03–현재

Generative UI를 단순 UI 생성이 아니라 여러 내부 시스템을 연결하고 조작하는 Agent 실행 레이어로 재정의했습니다.

- 사용자 자연어 요청을 기반으로 계정, 결제, 매핑 등 여러 내부 도구의 조회 순서를 구성
- Alibaba PageAgent를 확장해 Chrome Managed Profile 환경에서 탭 이동, click, input 등 실제 UI 조작 자동화
- 비결정적인 Generative UI 출력을 운영 도구에 직접 적용하기보다, 재현 가능한 action sequence와 결과 UI로 분리
- CS/QA 담당자가 여러 테이블을 직접 탐색하던 반복 조사 업무를 자동화 흐름으로 단순화

Vuddy 커머스 프론트엔드

2025.07–현재

Vuddy의 상품 탐색, 장바구니, 인증 등 커머스 프론트엔드를 지속적으로 운영하고 개선하고 있습니다.

- 장바구니와 인증 시스템 유지보수 및 개선
- Amplitude, ChannelTalk 기반 운영 흐름 관리
- Design System 기반 프론트엔드 패턴 정리

리디주식회사

프론트엔드 개발자
2022.05–2025.07

React 기반 웹, 모바일, 데스크탑 등 멀티 플랫폼 클라이언트 개발을 수행했습니다. MAU 213만 규모 콘텐츠 플랫폼에 서버 주도 UI(RiGrid)를 도입해 운영자가 UI를 직접 제어하고 실험할 수 있는 구조를 만들었습니다.

Next.js React TypeScript Emotion Jest PHP Twig Sentry Virtualization

리디 웹 플랫폼

2022.05–2025.07

RiGrid와 island architecture 마이그레이션에 참여했습니다.

- 서버 선언 UI를 클라이언트에서 해석
- PHP 페이지를 React 구조로 점진 이전
- A/B 테스트와 운영 UI 변경 유연화
- 모바일·데스크탑·특수 디바이스 안정성 유지

대규모 리스트 렌더링 최적화

2022.05–2025.07

6000개 이상 콘텐츠 리스트의 렌더링 병목과 체감 속도를 개선했습니다.

- 대량 DOM 렌더링의 프레임 드랍 분석
- windowing으로 화면 영역만 렌더링
- 스크롤 위치와 동적 높이 처리
- skeleton UI로 초기 로딩 경험 개선

사용자 환경 안정화

2022.05–2025.07

Sentry 오류 추적과 브라우저·디바이스 대응으로 사용자 환경을 안정화했습니다.

- 실제 사용자 오류를 Sentry로 추적
- 번역 플러그인 등 외부 extension 충돌 대응
- E-ink 렌더링과 UX 이슈 개선

티맥스엔터프라이즈

연구원
2020.02–2022.05

現 티맥스비즈에이아이
舊 티맥스데이터

AutoML 제품과 플랫폼 기획에 참여하고, AI 기능을 서비스 UI로 전달하는 인터페이스를 설계했습니다.

React · TypeScript · Material-UI · Sass · Python · jQuery

레거시 시스템 React 전환

2021.01–2022.05

사내 플랫폼 제품군을 jQuery 기반 UI에서 React로 이전했습니다.

- 기존 UI 코드를 용도별로 분리
- 레거시 위젯을 대체할 컴포넌트 신규 개발

AutoML 플랫폼

2020.02–2022.05

AutoML 플랫폼의 기능 흐름과 프론트엔드 인터랙션을 설계했습니다.

- 비전문가용 Codeless 개발 스튜디오 구현
- 설명 가능한 AI(XAI) 시각화와 대시보드 개발
- AutoML 엔진과 모델 개발자 간 인터페이스 연구
- 초기 제품의 요구사항과 구조 구체화

Education

울산과학기술원 (UNIST)

컴퓨터공학 석사
2018.03–2020.02
기술경영학 학사 (컴퓨터공학 융합전공)
2013.03–2018.02

AI 기반 UI와 추천 시스템 연구

2019.01–2020.02

추천 시스템에서 탐색 아이템 노출 방식이 사용자 로그와 피드백에 미치는 영향을 연구했습니다.

- Explore-Exploit 문제에서 투명성 효과 측정
- 탐색 아이템 노출 방식과 사용자 인식 분석
- 사용자 로그 가치를 평가하는 metric 제안
- 웹 기반 영화 추천 실험 시스템 구현
- 데이터 기반 UI 실험 설계

Python Flask Surprise jQuery

웹 사용자 행동 모델링

2019.02–2020.02

웹 환경에서 사용자 행동 패턴을 파악하고 모델링하는 연구를 수행했습니다.

- 역강화학습으로 사용자 보상 함수 추정
- 데이터 분석, 역사 교육, 카드 게임 환경을 다룸
- Tableau와 위키피디아 기반 교육 도구 활용

Python TensorFlow Keras scikit-learn

Publications

An Empirical Analysis on Transparent Algorithmic Exploration in Recommender Systems

김기환
A Computing Research Repository (CoRR), 2108.00151, 2021

- 추천 시스템에서 사용자 취향을 파악하기 위한 랜덤 아이템들을 어떻게 전달해야 할까?
- 실험 환경으로 사용된 웹 기반 영화 추천 시스템 구현
- 사용자 로그의 가치를 AI 측면에서 평가하는 metric 제안
- 추천시스템의 Explore-Exploit 문제에서 투명성이 데이터의 품질에 미치는 영향을 측정
- Amazon MTurk에서 94명의 피험자를 구인
- 피험자들에게 넷플릭스와 유사한 실험 환경을 이용하도록 하여, 실사용 로그 데이터와 설문 응답을 수집

ST-GRAT: A Novel Spatio-temporal Graph Attention Networks for Accurately Forecasting Dynamically Changing Road Speed

박천복, 이충기, 방효진, 태윤원, **김기환**, 진승민, 고성안, 주재걸
ACM International Conference on Information and Knowledge Management (CIKM), 2020

- 한국도로공사의 도로 바닥에 설치된 차량 감지 센서 데이터를 전처리
- Attention이 효과적으로 동작한 경우들을 패턴 별로 묶어 카테고리화

Modeling Exploration/Exploitation Decisions through Mobile Sensing for Understanding Mechanisms of Addiction

김기환, 김상훈, 이충기, 고성안
ACM International Conference on Mobile Systems, Applications, and Services (MobiSys), 2019

- Inverse Reinforcement Learning을 통해, 스마트폰 사용 로그로 중독 질환 여부를 감지하는 시스템을 제안

An Empirical Study on the Relationship Between the Number of Coordinated Views and Visual Analysis

오주영, 이충기, 김휘연, **김기환**, 권오상, Eric D. Ragan, 권범철, 고성안
A Computing Research Repository (CoRR), 2204.09524, 2018

- 시각화 차트의 갯수가 데이터의 시각적 분석에 어떤 영향을 미치는지 실험
- 44명 피험자에게 시각적 분석 도구를 주고, 데이터 분석 과제를 풀도록 함
- Think-aloud 프로토콜과 녹화된 화면, 그리고 로그 데이터를 통해 사용자의 분석 패턴을 카테고리화
- 차트의 갯수와 과제 점수 간 양의 상관관계를 관찰

시각화 기반 딥러닝 분석 기술

이재성, **김기환**, 이충기, 고성안
소음진동 제27권 제6호 2017.11

- 딥 러닝 모델을 해석하기 위한 시각화 기법들을 정리 및 조사

CASE STUDY · INTERNAL TOOLS · BROWSER AGENT

PageAgent 기반 내부 운영 자동화

Generative UI를 단순히 화면을 생성하는 기술이 아니라, 여러 내부 시스템을 연결하고 조작하는 Agent 실행 레이어로 재정의한 작업입니다.

PageAgent | Generative UI | Browser Automation | QA/CS Automation | Internal Operations

Problem

예를 들어 카카오 계정으로 구매한 사용자가 네이버 계정으로 로그인해 구매 내역이 보이지 않는 CS가 들어오면, 운영자는 여러 도구를 오가며 구매 귀속 계정을 추적해야 했습니다.

- 소셜 로그인 계정 간 구매 귀속 확인
- 유저, 결제, 계정 매핑 테이블의 순차 조회
- 배송 이슈에서는 택배사 외부 서비스까지 조회
- 운영자가 매번 조회 순서를 직접 판단해야 하는 부담

Initial Hypothesis

요청마다 확인해야 할 정보와 순서가 달라지기 때문에, 자연어 요청에 맞춰 조회 흐름과 UI를 동적으로 구성하면 탐색 업무를 줄일 수 있을 것이라 기대했습니다.

- 정적인 어드민 화면을 추가하는 대신 동적 조사 흐름 구성
- 계정 후보, 결제 내역, 매핑 결과를 한 화면에서 재구성
- 운영자가 다음에 볼 테이블과 순서를 직접 판단하는 부담 감소

Why Generative UI Was Not Enough

실제로는 UI를 새로 만드는 것보다 기존 사내 도구를 오가며 정확하게 조회하고 조작하는 능력이 더 중요했습니다.

정보가 여러 어드민과 탭에 분산	조회와 입력은 여전히 사람이 수행	핵심은 서비스 이동과 조작

Key Insight

> **내부 도구에서 중요한 것은 UI 생성이 아니라, 여러 시스템을 연결하고 조작하는 능력이다.**

Approach

PageAgent를 기반으로 사용자의 자연어 요청을 실행 가능한 action sequence로 변환하고, 브라우저에서 실제 UI를 조작한 뒤 결과를 다시 운영자가 이해하기 쉬운 형태로 재구성했습니다. 익스텐션 기반 구조를 통해 탭마다 분리된 브라우저 컨텍스트의 데이터를 Agent 레이어에서 이어 붙일 수 있었습니다.

Natural Language 운영자 요청 → **LLM** 의도 해석 → **PageAgent** UI 조작 실행 → **Browser** 여러 탭 이동 → **Result UI** 정보 재구성

Example Scenario

위와 같은 CS 조사 업무에서, Agent 적용 전후의 작업 흐름은 다음처럼 달라집니다.

Before

1. 문의 정보에서 후보 사용자 식별
2. 유저 어드민에서 소셜 계정 확인
3. 계정 매핑 테이블에서 동일 사용자 후보 탐색
4. 결제 어드민에서 구매 귀속 계정 조회
5. 필요 시 택배사 등 외부 서비스까지 조회
6. 여러 탭의 결과를 사람이 직접 종합

After

1. 운영자가 자연어로 조사 요청
2. Agent가 필요한 조회 순서 생성
3. PageAgent가 여러 탭에서 click/input 수행
4. 구매 귀속 계정과 현재 로그인 계정의 차이 요약
5. CS 답변에 필요한 결과 UI 제공

Architecture

- **LLM:** 사용자 명령을 해석하고 실행 가능한 작업 흐름 생성
- **PageAgent:** 브라우저 탭 이동, 버튼 클릭, 입력 등 실제 UI를 조작하고, 탭 간 분리된 데이터를 Agent 레이어에서 연결
- **Chrome Managed Profile:** 사내 도구 접근 권한과 인증 세션을 유지하고, 브라우저 익스텐션 형태로 Agent를 배포하는 채널
- **UI Layer:** 수집한 정보를 운영자가 판단하기 쉬운 결과로 재구성

Implementation Scope

PageAgent의 브라우저 조작 엔진을 새로 만든 것이 아니라, 내부 운영 도구에 맞는 통합 레이어와 업무 실행 흐름을 설계하고 검증했습니다.

- **Workflow Mapping:** CS/QA 요청을 PageAgent가 수행 가능한 action sequence로 변환하는 업무 흐름 정의
- **Selector Strategy:** 반복 사용 UI는 전용 selector로 연결해 시각 탐색 의존도를 낮추고 조작 안정성 강화
- **Extension Context:** 탭마다 분리된 브라우저 컨텍스트를 운영 업무 단위로 이어 붙이는 구조 검토
- **Result Contract:** 수집 근거와 요약 결과를 분리해 운영자가 검증 가능한 형태로 재구성

Impact

반복 조사 단순화

내부 어드민과 외부 서비스를 오가던 QA/CS 흐름을 자동화 대상으로 정리

기존 UI 유지

기존 어드민 위에 실행 레이어를 추가하는 방향 검증

재현성 강화

전용 selector로 시각 탐색 비용 축소

What I Learned

- Generative UI는 standalone으로는 내부 운영 도구의 문제를 해결하기 어렵다.
- Agent와 결합했을 때 실제 업무 자동화 가치가 커진다.
- 내부 도구에서는 정확성, 재현성, 감사 가능한 실행 흐름이 핵심이다.
- UI를 새로 만드는 것보다 기존 시스템을 안정적으로 연결하고 조작하는 능력이 더 중요하다.

CASE STUDY · PRODUCT UI PLATFORM · SERVER DRIVEN UI

RiGrid 기반 Server Driven UI

콘텐츠 플랫폼의 UI 구성과 비즈니스 데이터를 분리하고, Grid/Cell 단위로 재사용 가능한 클라이언트 렌더링 구조를 설계한 작업입니다.

Server Driven UI · RiGrid · GraphQL · Multi-platform UI

Problem

기존 콘텐츠 UI는 비즈니스 데이터와 표시 로직이 강하게 결합되어, 배지나 섹션 구성이 바뀔 때마다 여러 클라이언트 코드 수정이 필요했습니다.

- 플랫폼별로 유사한 UI 로직이 중복 구현됨
- 운영 실험이나 콘텐츠 구성을 바꾸기 위해 앱/웹 배포가 필요함
- 작품 상세, 랜딩, 검색처럼 서로 다른 화면에서 재사용 가능한 구조가 부족함

Approach

서버가 Grid와 Cell 구조를 내려주고, 클라이언트는 schema에 맞춰 Cell tree를 재구성한 뒤 대응되는 컴포넌트를 렌더링하는 구조로 접근했습니다.

- GraphQL schema 기반 Grid/Cell 인터페이스 정의
- Cell의 layout과 type으로 UI 표현과 데이터 바인딩을 분리
- AsyncCell, ChildCells, FlexColumnCell 등 유틸리티 Cell로 조합성 보강

Implementation Focus

Client Renderer

서버 응답의 Cell 목록을 트리로 재구성하고, layout/type에 맞는 컴포넌트로 매핑

Compatibility

기존 클라이언트가 깨지지 않도록 추가 중심의 schema 변경과 점진적 전환 고려

Operational Flexibility

Cell 단위 조합으로 랜딩/검색/상세 화면의 콘텐츠 구성과 실험 가능성 확대

Impact

- UI 데이터와 비즈니스 데이터를 분리해 변경 영향 범위를 줄이는 방향 검증
- 기획, 디자인, 개발이 Cell 단위 구조를 공유하며 협업할 수 있는 공통 언어 확보
- 마케팅 랜딩 페이지와 검색 페이지 등 다양한 화면으로 Server Driven UI 적용 범위 확장
- 부분 전환 과정에서 middleware 병목과 성능 저하를 확인하고, 전체 전환 필요성을 정리

Source: ridicorp.com/story/rigrid-server-driven-ui

CASE STUDY · FRONTEND PERFORMANCE · VIRTUALIZATION

대규모 콘텐츠 렌더링 최적화

예상보다 큰 콘텐츠 목록을 안정적으로 렌더링하기 위해 DOM 수, 네트워크 페이로드, 사용자 상호작용 지연을 함께 줄인 프론트엔드 성능 개선 작업입니다.

Virtualization · React · Performance · Large Lists

Problem

250개 작품 기준으로 만들어진 페이지가 실제 이벤트에서 6,000개 이상의 작품을 다루게 되면서 렌더링과 상호작용 성능이 급격히 나빠졌습니다.

- 대량 DOM 렌더링으로 저사양 기기에서 초기 표시가 지연됨
- 스켈레톤과 실제 아이템 렌더링 책임이 한 컴포넌트에 섞임
- 필터 변경 시 네트워크 재요청과 메인 스레드 부하가 함께 발생함

Approach

화면에 보이는 아이템만 렌더링하도록 리스트 가상화를 적용하고, skeleton 표현과 데이터 필터링 흐름을 분리해 체감 성능을 개선했습니다.

- window scroll과 결합한 list virtualization 적용
- SVG background 기반 skeleton으로 렌더링 책임 분리
- gzip 압축과 클라이언트 필터링 구조 검토

Implementation Focus

Rendering Budget

전체 DOM을 한 번에 그리지 않고 viewport 주변 아이템만 유지하도록 구조 변경

Skeleton Strategy

스켈레톤을 반복 background로 처리해 가상화 컴포넌트의 관심사를 단순화

Interaction Cost

필터링 중 perceived loading과 실제 데이터 처리 비용을 함께 분석하고 개선

Impact

- 대량 콘텐츠 목록에서도 초기 렌더링과 스크롤 체감 성능을 유지할 수 있는 구조 확보
- 스켈레톤 표시와 실제 아이템 렌더링을 분리해 컴포넌트 책임을 명확하게 정리
- 큰 JSON payload에 gzip을 적용해 전송 크기를 약 1/8 수준까지 줄일 수 있음을 확인
- 필터링 로직에서 네트워크 비용과 CPU 비용을 함께 봐야 한다는 기준을 정립

Source: ridicorp.com/story/ridi-markdown-improvements